IndividualProject Report

1. Introduction

The goal of this project is to build a CNN model for the SVHN dataset using TensorFlow/Keras. The dataset contains 32×32 color digit images from street view house numbers. I used the training dataset to train the model and the test dataset to evaluate the final performance.

2. CNN Structure and Hyperparameters

I used a CNN model because CNNs work well for image classification tasks. Convolution layers can extract image features such as edges, shapes, and digit patterns.

The model uses ReLU activation, max pooling layers, dropout layers, and batch normalization layers. Max pooling reduces the image size while keeping important features. Dropout and batch normalization help reduce overfitting and make training more stable.

Several hyperparameters were selected before training, like filter size, number of filters, batch size, learning rate, dropout rate, and number of epochs. I used 3×3 convolution filters and increased the number of filters from 32 to 64 and then 128 in deeper layers.

3. Code logic

The Python program first loads the SVHN training and testing datasets from the .mat files using scipy.io.loadmat. After loading the data, I changed the image shape from (32, 32, 3, N) to (N, 32, 32, 3) because TensorFlow/Keras requires the image number to be the first dimension.

Next, I reshaped the labels into one-dimensional arrays and changed all labels equal to 10 into 0 because SVHN uses label 10 to represent digit 0. The image values were normalized by dividing by 255. After preprocessing, the CNN model was trained using the training dataset.

The program also saves the accuracy curve, loss curve, confusion matrix, sample predictions, and feature map visualizations into the outputs folder.

4. Starting model and final CNN architecture

My starting model was based on a simple CNN structure similar to the MNIST CNN example. The first version only used a small number of convolution layers and one dense layer.

However, the SVHN dataset is more difficult because the images are colored and contain background noise. The first model could not achieve high enough validation accuracy.

To improve the model, I gradually added more convolution filters and dropout layers. Later, I added batch normalization layers and a third convolution block. These changes improved the validation accuracy and reduced overfitting.

The final model contains three convolution blocks with 32, 64, and 128 filters. After the convolution layers, the model uses a dense layer with 256 units and a final softmax output layer for digit classification.

Final architecture summary

Part	Layers	Purpose
Block 1	Conv2D 32, Conv2D 32, BatchNorm, MaxPool, Dropout	Learn simple edges and colors
Block 2	Conv2D 64, Conv2D 64, BatchNorm, MaxPool, Dropout	Learn digit stroke patterns
Block 3	Conv2D 128, Conv2D 128, BatchNorm, MaxPool, Dropout	Learn higher-level digit features
Classifier	Flatten, Dense 256, BatchNorm, Dropout, Dense 10	Make final digit prediction

5. Intermediate stages and feature visualization

I improved the CNN model step by step during the project. The first model was too simple and the validation accuracy was not high enough. After adding more convolution filters and dropout layers, the model performance improved. Batch normalization also made the training process more stable. The feature map visualizations show how the CNN extracts image features in different layers. Early layers mainly detect simple patterns, while deeper layers learn more complex digit features.

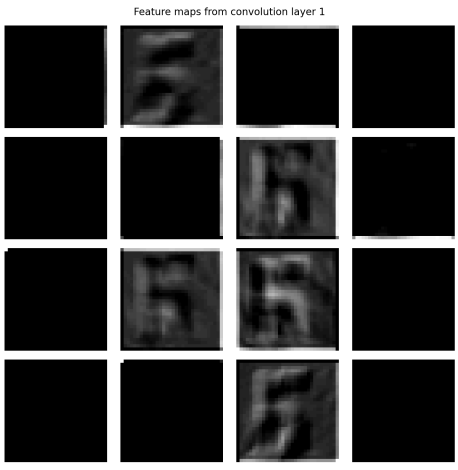


Figure 1. Feature maps from an early convolution layer.

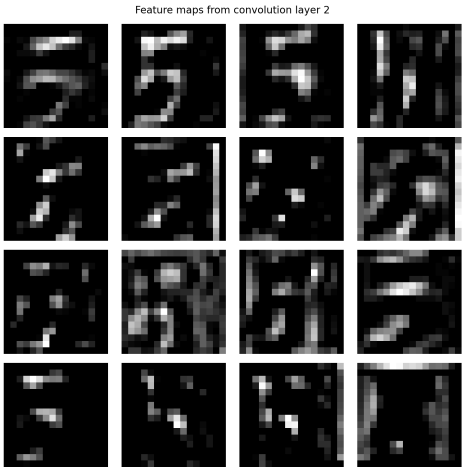


Figure 2. Feature maps from a middle convolution layer.

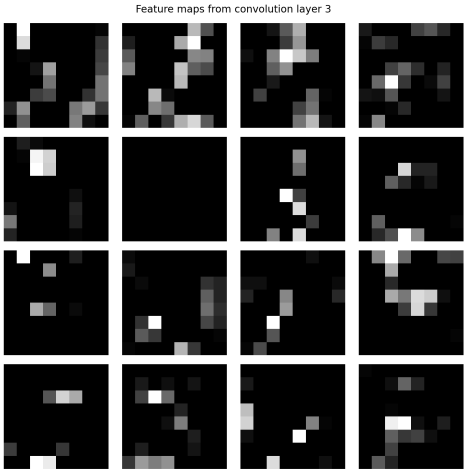


Figure 3. Feature maps from a deeper convolution layer.

6. Model evaluation

I trained the model with Adam optimizer and sparse categorical cross entropy loss. I used 10% of the training data as validation data. I also used callbacks during training.

ModelCheckpoint saved the best model. ReduceLROnPlateau reduced the learning rate when validation loss stopped improving. EarlyStopping helped avoid unnecessary training.

The model trained for 20 epochs. Near the end of training, the validation accuracy was around 95%. The final test result was:

Final test loss = 0.2253
Final test accuracy = 0.9592

This means the final test accuracy is about 95.92%, matching the requirement.

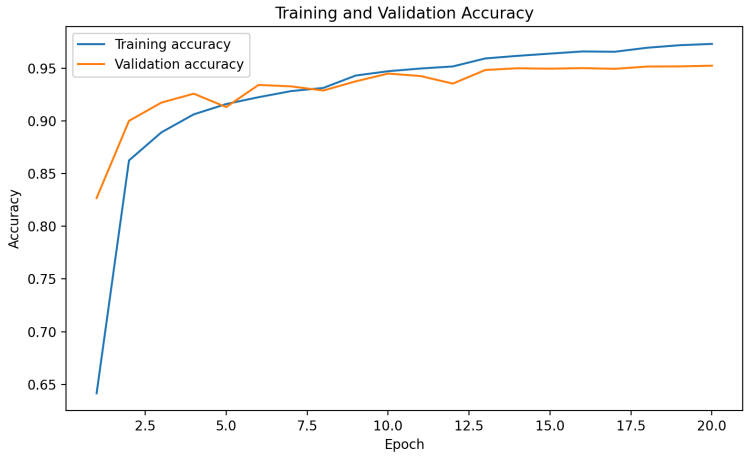


Figure 4. Training and validation accuracy.

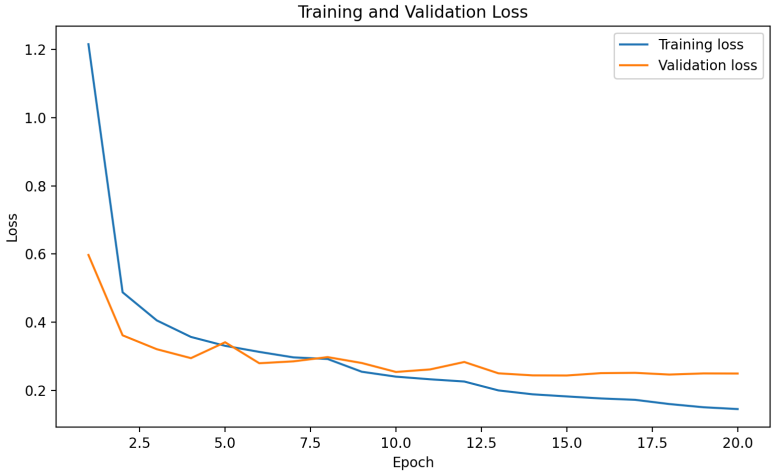


Figure 5. Training and validation loss.

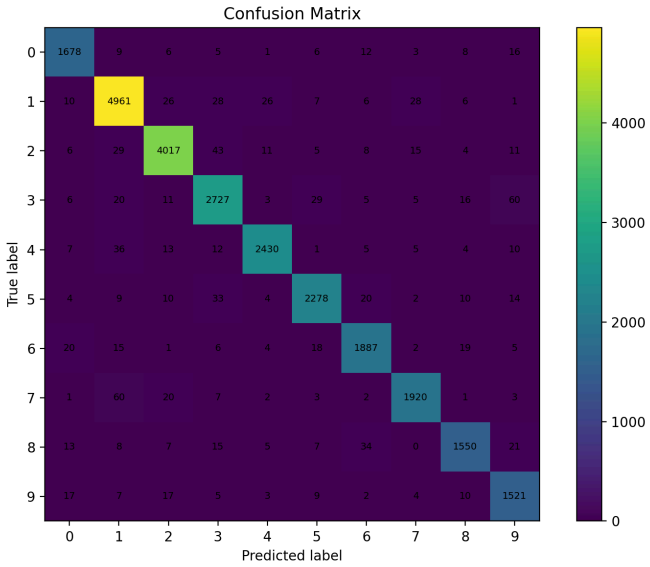


Figure 6. Confusion matrix on the test data.



Figure 7. Example test images with true and predicted labels.

The classification report also shows good performance. The overall accuracy is 0.96. Most digit classes have precision, recall, and f1-score around 0.95 or higher. This means the model performs well on most digits.

7. Difficulties

The first difficulty was the data format. The SVHN data is stored in .mat files. Also, the image shape is not the same as the Keras input shape, so I had to transpose the data.

The second difficulty was the label rule. In SVHN, label 10 means digit 0. This needed to be fixed before training.

The third difficulty was overfitting. A deeper CNN can fit the training data well, but it may not work well on test data. To reduce overfitting, I used dropout, batch normalization, validation split, and early stopping.

Another difficulty was training time. Each epoch took some time, so I had to wait for the model to finish training.

8. Possible improvements

One way is to use data augmentation, such as small shifts, rotations, and brightness changes. This can help the model learn more real-world image changes.

Another way is to try a deeper model. I could also tune the learning rate, batch size, and dropout rate more carefully.

If there is more training time, I can train the model with stronger data augmentation. This may improve the test accuracy.